ES 116
Principles and Applications of Electrical Engineering
Digital Circuits: Review

EE Department
IIT Gandhinagar

Book References

"Digital Systems" by Ronald J. Tocci, Prentice Hall, India
"Digital Fundamentals" by Thomas L. Floyd, Pearson Education.
"Digital Design" by M. Morris Mano, Pearson Education.

April 16, 2026

Recap . . .

- In the previous lecture, we had learnt about Boolean algebra, its axioms and theorems and their use in expressing and manipulating logic expressions.
- We had also learnt about Karnaugh maps – a graphical method for logic minimization.
- We had seen how logical expressions can be implemented using gates.

In these applications, the output of a circuit block was a Boolean function of the *instantaneous values* of inputs. Such circuits are known as **combinational** digital circuits.

In this lecture, we'll discuss circuits where a *sequence* of digital values is a function of a *sequence* of digital inputs.

Such circuits are known as **Sequential Digital Circuits**.

Sequential Digital Circuits: Requirements

If the input is a *sequence* of digital values, how do we access the *previous values* of an input?

To implement a sequential digital circuit,

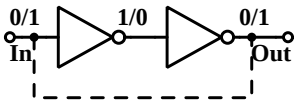
- The sequence of digital values should be finite.
- There should be a time marker, which will separate the previous value of a variable from its current value. If this marker occurs at regular intervals of time, it is called a **clock**.
- There should be a way to store the previous values of a variable, so that we can evaluate the output as function of the whole sequence. Elements which store these values are called **memory elements**.

Memory Elements

Sequential Digital Circuits require memory elements in addition to combinational logic functions. How can we implement memory?

- We need to make a copy of the input – so that we can use the copy instead of the original input after the input value has changed.
- We need an arrangement which stores this copy for future use.

Consider the buffer circuit shown on the right. The output is a copy of the input.
If we connect this output to the input, (as shown by the dashed line), it will retain its value for ever.

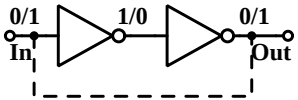


The circuit has two stable states. One is with input '0', intermediate node '1' and output '0'. The other is with input '1', intermediate node '0' and output '1'
Such circuits are called **bistable** circuits.

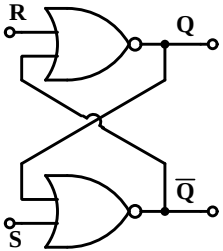
Memory Elements . . .

However, there is a problem with the buffer circuit with output connected back to input.
If the output is connected to the same node as the input, how do we change the state of this bistable circuit?

If the input and the output is '0' and we drive the input to '1', there will be a clash between the feedback and the input, and the circuit may malfunction.

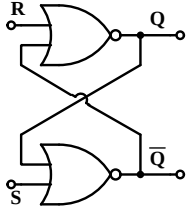


To solve this problem, we replace the inverters with 2 input inverting gates.
Now one input is used for connecting to the output, while the other is connected to the input.
The circuit on the right shows a cross connected pair of NOR gates as a memory element. It is known as the RS latch.



Memory Elements: RS Latch

Consider the cross connected NOR gates in the circuit shown below.



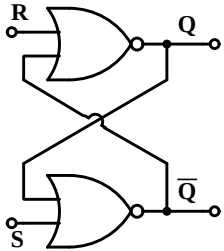
For R = '0' and S = '0', the circuit can remain in one of these two states:

1. Q = '0' and $\overline{Q} =$ '1', (reset state)
2. Q = '1' and $\overline{Q} =$ '0'. (set state)

Draw the wave form for Q and $\overline{Q}$,
if initially R and S = 0, Q = 0, $\overline{Q} = 1$;
now S goes to 1, then returns to 0, while R remains at 0.

Setting the RS flipflop

It is easy to see that for $R = 0, S \rightarrow$ '1' will force the circuit into its 'set' state (Q = 1, $\overline{Q}$ ='0').



$$S \rightarrow \text{'1'} \Rightarrow \overline{Q} = \text{'0'},$$

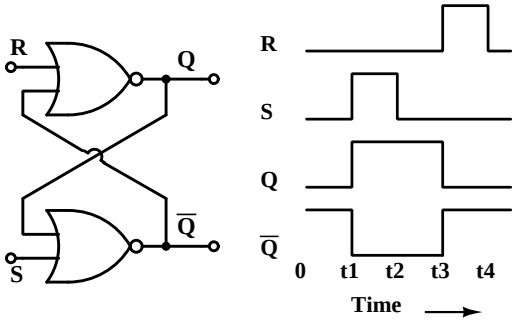
$$R = \text{'0'},\ \overline{Q} = \text{'0'} \Rightarrow Q = \text{'1'}$$

Now even if S returns to '0', the circuit will remain in its set state.

Thus this circuit "remembers" that S had been '1' in the past.

Similarly, R = '1' while S = '0' will force the circuit in its 'reset state' and it will remain in reset state even after R returns to '0'.

RS Latch: set and reset action



Initially R = S = '0' and the latch is in reset (Q = '0', $\overline{Q}$ = '1') state.

At t1, S goes to '1' and it forces $\overline{Q}$ to '0' unconditionally.
Since R = '0' and $\overline{Q}$ = '0',
Q goes to '1'.

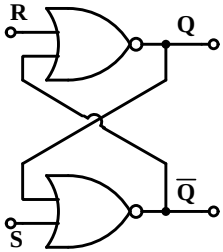
At t2, S returns to '0', and the latch remains in set (Q = '1', $\overline{Q}$ = '0') state.

At t3, R goes to '1'. It forces Q to '0' unconditionally.
Since S = '0' and Q = '0', $\overline{Q}$ goes to '1'.

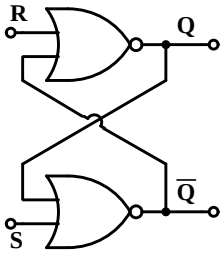
at t4, R returns to '0'. With R = 0, S = 0,
the latch remains in reset (Q = '0', $\overline{Q}$ = '1') state.

RS Latch: Race Condition

What will happen if R and S **both** become ‘1’?



RS Latch: set and reset action



Since the S input sets Q to ‘1’, it is called the ‘Set’ input.

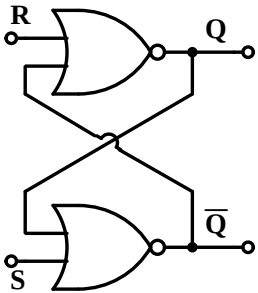
R input returns Q to ‘0’ and is termed as the ‘Reset’ input.

The latch retains its set (Q = ‘1’, $\overline{Q}$ = ‘0’) state even after S returns to ‘0’. Thus, it “remembers” that S had gone to ‘1’ before it returned to ‘0’.

Similarly, the latch retains its reset (Q = ‘0’, $\overline{Q}$ = ‘1’) state even after R returns to ‘0’. Thus it “remembers” that R had gone to ‘1’ before it returned to ‘0’.

RS Latch: Race Condition

What will happen if R and S **both** become '1'?



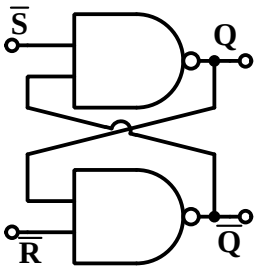
If R = S = 1, both Q and $\overline{Q}$ will be forced to '0'.
If R returns first to 0, we'll be in the set condition (R = 0, S = 1)
This will make Q = 1 and $\overline{Q} = 0$
Now when S also returns to 0, the latch will retain this state.

If S returns first to 0, we'll be in the reset condition (R = 1, S = 0).
This will make Q = 0 and $\overline{Q} = 1$
Now when R also returns to 0, the latch will retain this state.

So the eventual state of the latch depends on which input returns to 0 **last**.
This is known as a **Race Condition**

RS Latch with cross connected NAND gates

Remember the duality property of Boolean logic?



We can also construct an RS latch using cross connected NAND gates. Its operation is similar to the cross connected NOR.

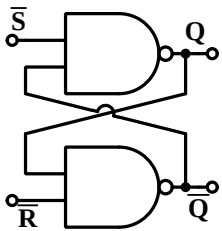
Here the circuit is idle when $\overline{R}$ and $\overline{S}$ inputs are at '1', while the 'set' and 'reset' action is triggered by $\overline{S}$ or $\overline{R}$ going to '0'.

Like the cross connected NOR circuit, Set and Reset are not supposed to go to their active level ('0' in this case) simultaneously.

Draw the wave form for Q and $\overline{Q}$, if initially $\overline{R}$ and $\overline{S} = 1$, Q = 0,$\overline{Q} = 1$.
Now $\overline{S}$ goes to 0, and then returns to 1, while $\overline{R}$ remains at 1.
Also, for the case when $\overline{R}$ and $\overline{S}$ are 1 initially, Q = 1,$\overline{Q} = 0$.
Now $\overline{R}$ goes to 0, and then returns to 1, while $\overline{S}$ remains at 1.

RS Latch in the Race Condition

Consider the cross connected NAND latch.



If both $\overline{R}$ and $\overline{S}$ inputs go to their active level ('0') simultaneously, Q as well as $\overline{Q}$ will go to '1'.

If $\overline{R}$ returns to 1 first, the latch will be in the set condition ($\overline{R} = 1, \overline{S} = 0$).
So the latch will be set. (Q = 1, $\overline{Q} = 0$).

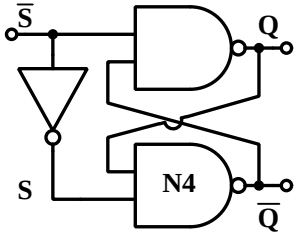
Similarly, if $\overline{S}$ returns to 1 first, the latch will be in the reset condition ($\overline{R} = 0, \overline{S} = 1$). Then the latch will be reset. (Q = 0, $\overline{Q} = 1$).

The output will settle to one of the two stable states depending on which of the two inputs is removed from the active state last. (**"Race Condition"**).

Improving the RS Latch

The Race Condition in an RS latch is a source of worry, since the output is unpredictable if both inputs ever become active simultaneously.
(The output depends on delays to the $\overline{R}$ and $\overline{S}$ inputs, which may vary)

Can we solve this problem by adding an inverter between $\overline{R}$ and $\overline{S}$ inputs, so that these are never active simultaneously?



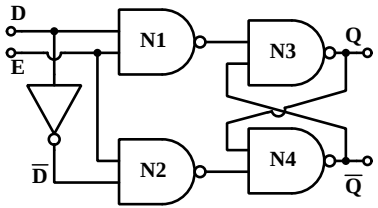
Unfortunately, this is does not lead to a useful circuit.
When $\overline{S}$ = 1, S = 0, $\overline{Q} = 1$, Q = 0
When $\overline{S}$ = 0, S = 1, Q = 1, $\overline{Q} = 0$

Therefore the whole circuit just reproduces S, ($Q = S$, $\overline{Q} = \overline{S}$), with no memory, since the output changes as soon as the input changes to either level.
(The NOR RS latch with an inverter has the same problem).

Improving the RS Latch . . . the D Latch

Adding an inverter between R and S can still be useful, if we can somehow stop the circuit from copying $\overline{R}$ to the output immediately. We can do this by inserting an enable signal E which controls the transfer of the input to the latch.

When E = 0, outputs of NANDs N1 and N2 are forced to '1', so the inputs to the NAND latch formed by N3 and N4 are at their passive level (1).
Thus the latch holds its old state.



When E = 1, $\overline{D}$ and D are applied to the latch.
this makes Q = D and $\overline{Q} = \overline{D}$

This circuit is called a D (or Data) latch. It copies its input data D to the output while the Enable input is high.
It retains this value when Enable goes low.
Then Q = previous value, D = current value and these can be different.

The Transparent D Latch

The D latch we have discussed has the property that,

Q = D, $\overline{Q} = \overline{D}$ when E = 1, and
Q = Q_{prev} and $\overline{Q} = \overline{Q_{prev}}$ when E = 0,
where Q_{prev} and $\overline{Q_{prev}}$ are the values of D and $\overline{D}$ when E went from 1 to 0.

This is known as a “transparent” D latch,
because Q follows D when E = 1 (transparent), and
when E = 0, Q “remembers” the value of D at the instant when E went from 1 to 0.

The latch is “transparent” when E = 1,
Latches when E goes from $1 \rightarrow 0$,
Retains latched value when E = 0.

Problem with Transparent Latches

- Consider a case when the D input to the latch is derived from its Q output. (D = f(Q))
- When the latch is transparent, it forces Q to the value of D. (Q = D).
- This may not be consistent with D = f(Q), and may lead to malfunction.

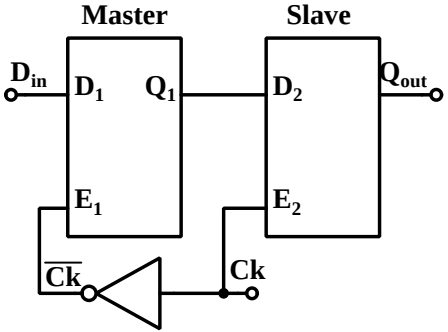
In most cases, we need a replica of the previous data as a snapshot taken at the instant of a clock transition, and no transparency.

- If the snapshot is taken at the positive transition of the clock ($0 \rightarrow 1$), the circuit is said to be positive edge triggered.
- If the snapshot is taken at the negative transition of the clock ($1 \rightarrow 0$), the circuit is said to be negative edge triggered.

Edge triggered memory elements (called flipflops) can be implemented using two transparent D latches enabled by complementary signals.

Edge triggered D flipflop

Consider two transparent D latches made from NAND gates as discussed earlier. These are latched when $E_1/E_2 = 0$ and transparent when $E_1/E_2 = 1$



When the clock is low, the first D latch is transparent. Output of the second latch is its previous value. $Q_1 = \overline{D}_{in}$. $Q_2 = Q_{2prev}$.

When the clock goes high, the first D latch captures the value at this instant, the second latch becomes transparent and outputs the value latched by the first D latch.

Thus, at any time, one of the two latches is non-transparent and the output does not follow the input. The output holds the value seen at D_{in} when the clock transitions from $0 \rightarrow 1$.

Edge triggered D flipflop . . .

The arrangement of two latches such that they are not transparent in the same phase of clock is known as a master-slave combination.

Sensitivity Alert!

These days, there is some sensitivity to the historically established terminology such as master-slave due to its association with slavery trade.

Many people use Lead-Follow or Controller-Controlled terminology instead of master-slave.

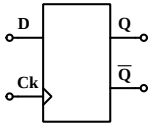
However, the term master-slave flipflops is still used quite widely.

Take your pick between being politically correct or being part of the mainstream usage . . .

We'll take the safe option of refering to these as edge-triggered!

Flipflops as storage circuits: D flipflop

Flipflops are used in sequential circuits with a clock.
Their action occurs whenever the clock has a specified transition, say from '0' to '1'. This transition is known as the **active edge of the clock**.

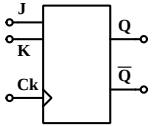


D flipflop: This is circuit with a 'D' (Data) input and a clock. At the active clock edge, it copies the value of D to its Q output and the complement appears at $\overline{Q}$.

The symbol for a D flipflop is shown above. The little triangle at the clock input shows that the flipflop transfers D to Q on the rising edge of the clock. A little circle is placed within the triangular symbol if the active edge is the falling edge of the clock.

Many circuit diagrams omit the indication for the active edge.

Other circuits with memory: JK flipflop



JK flipflop:
This circuit has two inputs J and K –
These act similarly to S and R inputs in the RS latch.

At the active edge of the clock,
If J = '0' and K = '0', the flipflop outputs remain unchanged.
If J = '1' and K = '0', Q goes to '1' and $\overline{Q}$ goes to '0'.
If J = '0' and K = '1', Q goes to '0' and $\overline{Q}$ goes to '1'.
If J = '1' and K = '1', it complements the values at Q and $\overline{Q}$.
(This and the presence of clock distinguish it from RS latch).

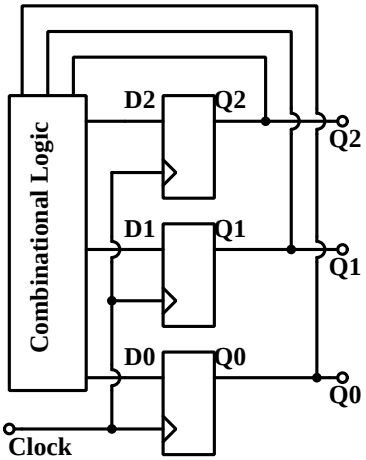
Toggle flipflop: In this, the J and K inputs are tied together and are used as the T input.
When T = '0', the flipflop outputs Q and $\overline{Q}$ remain unchanged.
When T = '1', it toggles *ie* complements the values at Q and $\overline{Q}$ at the active clock edge.

Sequential Circuit Example: a 3 bit Counter

Suppose we need a circuit which counts up at the arrival of each clock edge from 0 to 7 and then starts from 0 again.

(That is, in binary, it has the sequence of outputs 000, 001, 010, 011, 100, 101, 110, 111, 000 . . .)

This can be implemented by the scheme shown on the right.
We have to evaluate the functions which will generate the desired D values, so that the next count is placed at the D inputs before the next clock arrives.



Sequential Circuit Example: a 3 bit Counter

The combinational logic required for the counter has the following truth table:

Q2	Q1	Q0	D2	D1	D0
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	0

SOP expressions for D2, D1 and D0 are:

$$
\begin{aligned}
D2 &= \overline{Q2} \cdot Q1 \cdot Q0 + Q2 \cdot \overline{Q1} \cdot \overline{Q0} \\
&\quad + Q2 \cdot \overline{Q1} \cdot Q0 + Q2 \cdot Q1 \cdot \overline{Q0} \\
D1 &= \overline{Q2} \cdot \overline{Q1} \cdot Q0 + \overline{Q2} \cdot Q1 \cdot \overline{Q0} \\
&\quad + Q2 \cdot \overline{Q1} \cdot Q2 + Q2 \cdot Q1 \cdot \overline{Q0} \\
D0 &= \overline{Q2} \cdot \overline{Q1} \cdot \overline{Q0} + \overline{Q2} \cdot Q1 \cdot \overline{Q0} \\
&\quad + Q2 \cdot Q1 \cdot \overline{Q0} + Q2 \cdot Q1 \cdot \overline{Q0}
\end{aligned}
$$

3 bit Counter: Karnaugh Map minimization

For D2 **Q2Q1 ►**

Q0 ▼	00	01	11	10
0	0	0	1	1
1	0	1	0	1

For D1 **Q2Q1 ►**

Q0 ▼	00	01	11	10
0	0	1	1	0
1	1	0	0	1

For D0 **Q2Q1 ►**

Q0 ▼	00	01	11	10
0	1	1	1	1
1	0	0	0	0

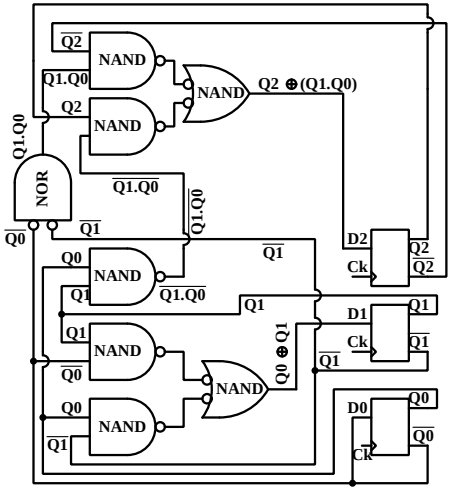
From the Karnaugh Maps:

$$
\begin{aligned}
D2 &= Q2 \cdot \overline{Q0} + Q2 \cdot \overline{Q1} + \overline{Q2} \cdot Q1 \cdot Q0 \\
&= Q2 \cdot (\overline{Q0} + \overline{Q1}) + \overline{Q2} \cdot Q1 \cdot Q0 \\
&= Q2 \cdot (\overline{Q0 \cdot Q1}) + \overline{Q2} \cdot Q1 \cdot Q0 \\
&= Q2 \oplus (Q1 \cdot Q0)
\end{aligned}
$$

$$
\begin{aligned}
D1 &= Q1 \cdot \overline{Q0} + \overline{Q1} \cdot Q0 \\
&= Q1 \oplus Q0
\end{aligned}
$$

$$
D0 = \overline{Q0}
$$

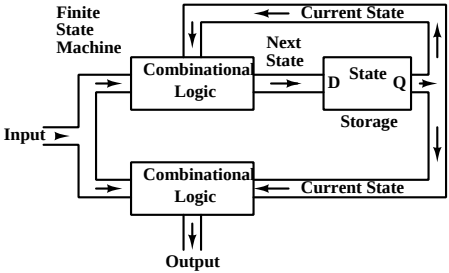
3 bit Counter: Logic Implementation



Finite State Machines

In a sequential circuit, the output sequence is a function of input sequence.
The sequences between which we want to establish a functional relationship have to be of finite length to be implementable.

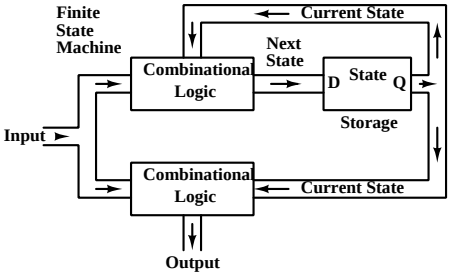
- A new set of input values are presented during each new clock period.
- It is not necessary to store all the past values of input variables.



A (possibly multi-bit) **state** is stored, which is a digital value with sufficient information about the past inputs so that the current element of the output sequence can be generated as a **combinational** function of the state and the current inputs.

Finite State Machines

- In each clock period, as fresh inputs arrive, a new value of **state** is generated which is a combinational function of the current value of state and current inputs.
- The state is thus a recursive function of previous inputs.

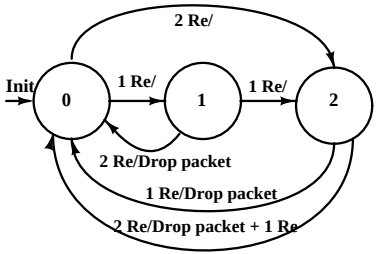


The **state** represents the "history" of input variables as relevant to production of the output sequence.

Systems which use this approach to generate sequential logic are called **Finite State Machines** or FSMs.

FSM example: Penny in the slot machine

Consider a Penny in the slot machine which accepts 1 Re and 2Re coins. It dispenses a packet worth 3 Rs. and returns coins if required.

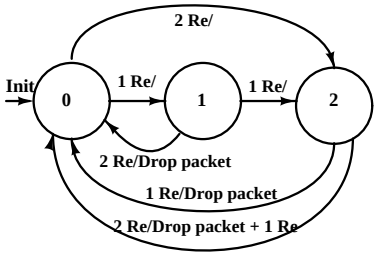


The state diagram on the left shows the evolution of 'states' in response to a sequence of inputs.

- At any time, one of two inputs may occur – insertion of a 1 Re coin or insertion of a 2 Re coin.
- In response, the machine may – i) do nothing, ii) Drop the packet or iii) Drop the packet and a 1 Re coin.

The machine can be designed with 3 states and combinational logic to determine the action and the next state in response to specific inputs.

FSM example: Penny in the slot machine



At power on, the machine wakes up in state 0.

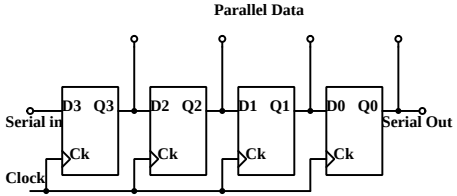
- At any time, The machine is in one of the three states shown.
- In any state, only two external inputs are possible: Arrival of a 1 Re coin or arrival of a 2 Re coin.

Curr st	Input	Output	Next St	Input	Output	Next St
0	1 Re	Nothing	1	2 Re	Nothing	2
1	1 Re	Nothing	2	2 Re	Drop packet	0
2	1 Re	Drop packet	0	2 Re	Drop packet + 1 Re	0

We can implement this with 2 flipflops to encode the state number and combinational logic to determine next state and output from current input and state.

Registers

- A group of D flipflops in parallel can be used to store a multi-bit value. This is called a **register**.
- We can also connect D flipflops in series as shown below. This arrangement is called a **Shift Register**.



At each active edge of the clock, $Q0 \rightarrow$ Serial Out, $Q1 \rightarrow Q0$, $Q2 \rightarrow Q1$, $Q3 \rightarrow Q2$, Serial In $\rightarrow Q3$.
Thus data is shifted to the right by one location.

- This can be used for serial to parallel conversion by feeding data in serially and after all data has been fed, collecting it in parallel.
- Shift register are also used for parallel to serial conversion. Data is loaded in parallel to all the D flipflops and shifted out serially.
- These are also used for data delay, as a serial memory etc.

Number Systems

- We have already seen how we can represent natural numbers in a base-2 system (binary representation).
  The bit pattern $b_{n-1} \ldots b_i \ldots b_1 b_0$ represents the number $N = \sum_{i=0}^{n-1} 2^i \cdot b_i$.
- Given a decimal number N, how do we determine b_i?
  - Take N and divide by 2. The remainder is b_0.
  - Repeat this process by dividing the quotient by 2 and retaining the remainder as $b_1, b_2 \ldots$. Continue till the quotient becomes 1 or 0. This is then the most significant bit.
  - For somewhat larger numbers, it is more efficient to convert the number to base-16 or Hex format, by using the above procedure but dividing by 16 every time.
  - The resulting Hex number can be converted to binary easily by replacing each hex digit by its binary equivalent.

An n bit binary number can represent any natural number in the range $0 \leq N \leq 2^n - 1/$

Representation of Natural numbers

Let us illustrate the conversion procedure by a few examples.

Given $N = 55$.
Divide successively by 2.

$55 = 27 * 2 + 1$, so $b_0 = 1$
$27 = 13 * 2 + 1$, so $b_1 = 1$
$13 = 6 * 2 + 1$, so $b_2 = 1$
$6 = 3 * 2 + 0$, so $b_3 = 0$
$3 = 1 * 2 + 1$, so $b_4 = 1$
Quotient = 1, so $b_5 = 1$

Thus decimal 55 = binary 111011

Given $N = 1000$. Divide by 16.
$1000 = 62 * 16 + 8$,
so least significant hex digit is 8.
$62 = 3 * 16 + 14$,
so the next digit is 14 or E.
$3 < 16$, so the most significant digit is 3. Thus 1000 = 3E8 in Hex.
Expanding each hex digit to binary, this can be written as:

0011 | 1110 | 1000.

Binary addition

- Binary numbers can be added just like decimal numbers, with a carry being transferred to a more significant bit position whenever the sum exceeds 1.
- We can represent positive numbers between 0 and $2^n - 1$ using n bits.
- What happens if the sum exceeds the largest representable number?

Consider an example using 4 bit numbers. Using 4 bits we can represent numbers between 0 and 15

Now consider the addition of 11 to 7 in binary arithmetic using 4 bits.
1011 + 0111 = 10010. The overflowing 5th bit can be used to signal that overflow has occurred and the result is not valid.

However, this provides us with a clue to a method for representing negative numbers.

Representing Negative numbers

- In arithmetic, we define a negative number by the property: $x + (-x) = 0$.
- To represent negative numbers, we take the magnitude of the number and then choose a number which when added to it will give zeros in the defined bit width, (albeit with an overflow).
- For example, the decimal number 5 is 0101 using 4 bit representation. When we add decimal 11 to it we get 0101 + 1011 = 10000. (5+11=16). If we ignore the overflow (fifth bit), the result is zero.
- Thus, if x = 0101, (-x) = 1011. Now, x + (-x) = 0, if we ignore the overflow.
- Obviously, we cannot use 1011 to represent 11 as well as -5!
  So we adopt a convention where numbers with 0 in the most significant position are positive, while numbers with 1 in the most significant position represent negative numbers.

Representing Negative numbers

- We use the convention where numbers with 0 in the most significant position are positive, while numbers with 1 in the most significant position represent negative numbers.
- With this convention, signed 4 bit numbers can be represented in the range -8 to + 7, inclusive of Zero.
- How do we find the number which when added to x will give zero (with overflow)?
- Notice that when we add a number to its complement (changing all 0's to 1 and 1's to zero), the sum will have 1 in all positions.
  Now if add 1 to this, we'll get all zeros with an overflow.
- So to get the negative of a number, we take its complement (also called 1's complement) and add 1 to it. (This is called 2's complement).

Representing Fractional Numbers

Recall the problem which we had solved during the tutorial . . .
Let us take a similar example where we want to represent 17.65 with 5 bits for the integral part and 8 bits for the fractional part.
We first represent 17 (5 bits) using successive division by 2.

Dividend	Quotient	Remainder
17	8	1
8	4	0
4	2	0

Dividend	Quotient	Remainder
2	1	0
1	0	1

So 17 base 10 = 10001 base 2

Representing the Fractional Part

There is an implied binary point after 5 bits, and bits to the right of it have place values which are negative powers of 2. Let us convert the fractional part (0.65) to the binary fractional part.

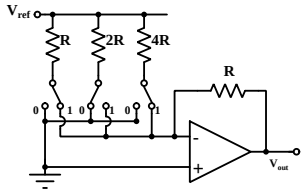
Multiplicand	Fraction	Integer
0.65	0.3	1
0.3	0.6	0
0.6	0.2	1
0.2	0.4	0

Multiplicand	Fraction	Integer
0.4	0.8	0
0.8	0.6	1
0.6	0.2	1
0.2	0.4	0

So 17.65 will be represented as 10001.10100110.
How will you represent -17.65?

Digital to Analog Conversion

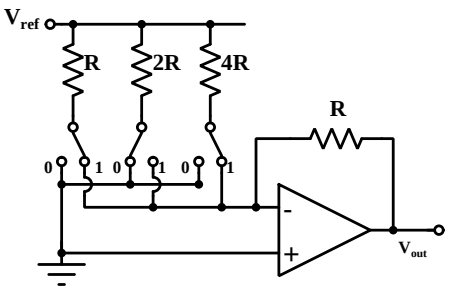
Consider the production of an analog quantity – say voltage, proportional to a given digital numbers. This is Digital to Analog conversion or DAC.



A digital number: $b_{n-1} \ldots b_i \ldots b_1 b_0$
represents $\sum_0^{n-1} 2^i b_i$.
If we use resistors with resistance values proportional to 2^i and use these to draw current from a reference voltage V_{ref} (with the other end of the resistors at ground potential), the currents will be in binary ratio.

We put a two way switch at the end other than V_{ref} which connects either to ground or to the virtual ground of an opamp.
Since these two choices are at the same potential, the current remains unchanged if we switch it between either of these.

Digital to Analog Conversion



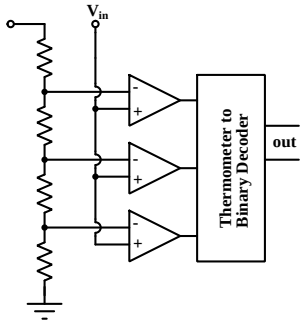
Each two way switch is controlled by a bit of the digital number which we want to convert. Since the two way switches direct the current either to ground or to the virtual ground of an opamp, the current through the resistors remains unchanged if we switch it between either of these.

The current going into the inverting terminal is then proportional to the digital number. The output voltage is $-R$ times this current and thus proportional to the digital number.

This method of D to A conversion is intuitive, but getting accurate values of resistors over a wide range is a difficult task.

Analog to Digital Conversion

A flash ADC is a fast converter which uses a separate comparator for each digital value possible. A resistor divider creates equally spaced N-1 reference levels.



- A bank of comparators compares the input to each of the reference levels.
- All comparators with reference level below the input value go to '1' while all those above it go to '0'.
  This is known as **Thermometer coding**.
- A decoder is then used to transform the thermometer code to binary.

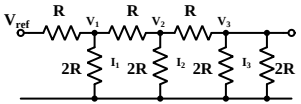
Flash converters are used for A to D conversion with a small number of bits, since its complexity grows exponentially with each additional bit.

Analog to Digital Conversion

- Many other types of A to D converters are available, providing a compromise between speed, power and complexity.
- A successive approximation ADC uses a DAC whose Digital input is provided by a register. The ADC sequentially adjusts the bits of this register and compares the output of the DAC with the input voltage, till they match. This type of ADC is widely used in medium speed applications.
- A dual slope ADC provides low complexity at low conversion rates. It converts its input voltage to a time interval, which is measured using a counter. This kind of ADC is used by most DMMs.

From the user's view point, a "start conversion" signal is sent to the ADC (of whatever type) and when the conversion is complete, it sends an "end of conversion" signal. The output of the ADC can be read as a digital number once the conversion is complete.

DAC with R-2R ladder network



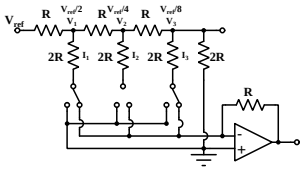
Consider a network with resistors of value R in series and those of value 2R in parallel in a ladder network as shown on the left.

Notice the extra 2R resistor at the end of the chain.

- At the end, we have 2 resistors to ground, each of value 2R. This is equivalent to a single resistor of value R.
- This equivalent resistance of R and the series resistance R from V_2 form a potential divider such that $V_3 = V_2/2$.
- The series resistor from V_2 and the equivalent resistor R to ground provide a total resistance of 2R to ground from V_2.
- Extending this argument, $V_2 = V_1/2$ and $V_1 = V_{ref}/2$
- Correspondingly, the current to ground through the 2R resistors is in binary ratio.
- by inserting a two way switch at the ground end and switching between ground and virtual ground, we can generate a voltage proportional to the digital value.

DAC with R-2R ladder network

The Circuit below shows a D to A converter using the R-2R ladder network.



- Bits of the digital value choose whether the current through the 2R resistors will go to ground or to virtual ground.
- Total current going into the virtual ground is converted to a voltage using a feedback resistor.

- Implementing the DAC is easier this way, because it requires only two resistance values. In fact, two identical resistors are put in series to get the 2R value.
- Accurate matching of resistors is much easier here compared to the binary weighted resistor case.

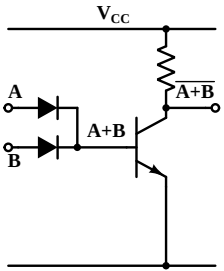
ADC with successive approximation

- This is a widely used architecture for ADCs.
- It uses a DAC and a comparator.
- At the first step, the most significant bit for the DAC is set.
- The comparator indicates if the unknown voltage is above or below the DAC output. If the unknown value is below the DAC value, the bit which was last set is cleared.
- Now the next significant bit for the DAC input is set. The unknown voltage is compared to the DAC output again.
- This process is continued till the least significant bit has been determined.

Logic families: DTL

Earliest implementation of Digital logic used relays connected in series/parallel to implement logic.

With the advent of semiconductor devices, diodes and transistors were used to implement digital logic. The circuit on the right is a DTL NOR gate.
A HIGH voltage (close to V_{CC}) represents a logic '1', while a LOW voltage (close to ground) represents a logic '0'.



When either A or B (or both) are HIGH, base current flows, saturating the bipolar transistor. This pulls the output LOW.
The transistor is OFF when both A and B are LOW as there is no base current. The output is then connected to V_{CC} through the collector resistor and is HIGH.

Logic families: TTL and CMOS

- Diode-Transistor Logic was eventually replaced by Transistor-Transistor Logic or TTL. This was the dominant digital technology for implementation of digital logic for a long time.
- TTL replaces the diodes at the input by a bipolar transistor with multiple emitters. In this course, we shall not get into the internal circuitry of TTL.
- Modern digital circuits use MOS transistors. By using complementary MOS transistor types (n channel and p channel), high speed can be obtained at reasonably low power consumption. This is known as CMOS logic.
- With advances in integrated circuit technology, multiple logic functions (and indeed entire micro-computers) can be placed on a single integrated circuit.